

PROJEKTOVANJE SLOŽENIH DIGITALNIH SISTEMA

**Hardverski akcelerator za prepoznavanje
šahovskih figura zasnovan na normalizovanoj
unakrsnoj korelaciji**

RT model, ASMD dijagram i analiza posle sinteze

Grupa: y25-g10

Sadržaj

1. Uvod

- 1.1. Predmet i obim dokumenta
- 1.2. Polazna tačka: ESL model sistema
- 1.3. Ciljna platforma i korišćeni alati

2. Opis algoritma

- 2.1. Matematička definicija
- 2.2. Optimizacija 1: kvadrirana korelacija (eliminacija korena)
- 2.3. Optimizacija 2: integralna slika (SAT)
- 2.4. Optimizacija 3: egzaktna celobrojna Q1.31 podela
- 2.5. Bitska analiza širina signala
- 2.6. Promenljiva veličina šablona

3. Uklanjanje petlji

- 3.1. Izgradnja integralne slike
- 3.2. Srednja vrednost šablona
- 3.3. Glavna petlja

4. Interfejs i okruženje

- 4.1. Četiri RT interfejsa
- 4.2. Registarska mapa
- 4.3. Portovi RTL jezgra
- 4.4. Memorijsko okruženje

5. ASMD dijagram

- 5.1. Pregled stanja
- 5.2. Faza A: učitavanje slike i izgradnja SAT
- 5.3. Faza B: šablon i njegova srednja vrednost
- 5.4. Faza C: priprema prozora
- 5.5. Faza D: pipeline-ovana MAC petlja
- 5.6. Faza E: normalizacija i upis rezultata

6. Datapath i controlpath

- 6.1. Registri
- 6.2. Funkcionalne jedinice
- 6.3. Sekvencijalni delilac
- 6.4. Memorije
- 6.5. Blok dijagram

7. Analiza posle sinteze

- 7.1. Metod
- 7.2. Utrošeni resursi

7.3. Kritična putanja i maksimalna frekvencija

7.4. Latencija i propusnost

7.5. Funkcionalna verifikacija

8. Zaključak

1. Uvod

1.1. Predmet i obim dokumenta

Projekat realizuje hardverski akcelerator za prepoznavanje šahovskih figura sa fotografije table. Slika table dimenzija 720×720 piksela deli se na 64 polja dimenzija 90×90 piksela; za svako neprazno polje računa se normalizovana unakrsna korelacija (engl. *Normalized Cross-Correlation*, NCC) sa šablonima figura, a iz mape skorova se rekonstruiše pozicija u standardnoj Forsyth-Edwards (FEN) notaciji. Korelacija je, prema profilisanju izvršne specifikacije, uzročnik preko 70 % procesorskog vremena i time prirodan kandidat za akceleraciju u programabilnoj logici.

1.2. Polazna tačka: ESL model sistema

Sistem je prethodno modelovan na sistemskom nivou (ESL) u okruženju SystemC/TLM-2.0 i dokumentovan u okviru predmeta *Projektovanje elektronskih uređaja na sistemskom nivou*. Taj model definiše partitionisanje hardvera i softvera, adresnu mapu, registarski interfejs akceleratora i vremenski budžet, i u ovom projektu se koristi kao **izvršna specifikacija**: RTL model mora dati bit-identičan rezultat.

Iz ESL modela se preuzimaju:

- bitske širine svih signala u datapath-u (ESL dokumentacija, Tabela 2), realizovane u odeljku 2.5;
- registarska mapa i adresni prostor programabilne logike (odeljak 4.2).

1.3. Ciljna platforma i korišćeni alati

Ciljna platforma je razvojna ploča **Zybo Z7-10** sa uređajem **xc7z010-clg225-2** (Zynq-7000). Uređaj raspolaže sa 17.600 LUT, 35.200 flip-flopova, 80 DSP48E1 blokova i 60 blokova RAMB36. Pretpostavljeni takt programabilne logike je 100 MHz, odnosno period 10 ns.

RT model je opisan u jeziku **VHDL-2008** i razvijen ručno iz ASMD dijagrama, bez visokonivoeke sinteze. Simulacija je izvršena alatom **XSim**, a sinteza i vremenska analiza alatom **Vivado 2025.2**.

2. Opis algoritma

Akcelerator prima segment slike dimenzija do 90×90 piksela i šablon figure dimenzija do 30×30 piksela, i za **svaku** poziciju prozora (u, v) računa kvadrat normalizovane unakrsne korelacije. Rezultat je mapa skorova dimenzija (img_w - tmp_w + 1) × (img_h - tmp_h + 1), koju softver zatim pretražuje tražeći maksimum.

2.1. Matematička definicija

Koeficijent korelacije γ u tački (u, v), za segment slike f i šablon t , definisan je izrazom:

$$\gamma(u,v) = \frac{\sum_{x,y} [f(x,y) - \bar{f}_{u,v}] \cdot [t(x-u, y-v) - \bar{t}]}{\sqrt{\sum_{x,y} [f(x,y) - \bar{f}_{u,v}]^2 \cdot \sum_{x,y} [t(x-u, y-v) - \bar{t}]^2}}$$

gde je $f(x,y)$ intenzitet piksela segmenta, $\bar{f}_{u,v}$ srednja vrednost piksela onog dela slike koji se trenutno preklapa sa šablonom, t piksel šablona, a $\bar{t}$ srednja vrednost celog šablona. Normalizacija oduzimanjem srednjih vrednosti čini meru neosetljivom na promenu osvetljenja, što je i razlog za izbor NCC-a umesto obične korelacije.

Direktna implementacija ovog izraza je za hardver neprihvatljivo skupa: sadrži kvadratni koren, deljenje po svakoj poziciji prozora i ponovno sumiranje prozora u svakoj tački. Sledeća tri odeljka opisuju tri transformacije kojima se izraz svodi na oblik pogodan za realizaciju u programabilnoj logici, **bez gubitka tačnosti**.

2.2. Optimizacija 1: kvadrirana korelacija (eliminacija korena)

Rezultat korelacije se ne koristi kao brojna vrednost, već isključivo za poređenje sa pragom detekcije: figura je prepoznata ako je $\gamma > 0,75$. Pošto je $\gamma \in [-1, 1]$ i pošto se traži maksimum po apsolutnoj vrednosti, poređenje se ekvivalentno može izvesti nad **kvadratom** korelacije:

$$\gamma^2(u,v) > 0,75^2 = 0,5625$$

Time kvadratni koren potpuno nestaje iz datapath-a, a izraz koji hardver računa postaje:

$$NCC^2(u,v) = \frac{(\sum \text{diff_f} \cdot \text{diff_t})^2}{\sum \text{diff_f}^2 \cdot \sum \text{diff_t}^2}$$

gde su $\text{diff_f} = f(x,y) - \bar{f}$ i $\text{diff_t} = t(x,y) - \bar{t}$. Umesto korena i jednog deljenja po tački, ostaju tri akumulacije i jedno deljenje.

2.3. Optimizacija 2: integralna slika (SAT)

Srednja vrednost prozora $\bar{f}_{u,v}$ zavisi od pozicije prozora, pa bi je naivna implementacija računala iznova za svaku poziciju — to je $\text{tmp_w} \times \text{tmp_h}$ sabiranja po **svakoj** od nekoliko hiljada pozicija. Umesto toga, jednom po segmentu se gradi **integralna slika** (engl. *summed-area table*, SAT), u kojoj svaki element sadrži sumu svih piksela iznad i levo od sebe:

$$\text{SAT}[y+1][x+1] = \text{SAT}[y][x+1] + \text{row_sum}(y, 0..x)$$

Suma proizvoljnog pravougaonog prozora tada se dobija u konstantnom vremenu, sa četiri pristupa memoriji:

$$\text{sum_f}(u,v) = \text{SAT}[v+th][u+tw] - \text{SAT}[v][u+tw] - \text{SAT}[v+th][u] + \text{SAT}[v][u]$$

Složenost pripreme je $O(\text{img_w} \times \text{img_h})$ i plaća se jednom po segmentu, dok se trošak po prozoru svodi sa $O(\text{tmp_w} \times \text{tmp_h})$ na $O(1)$. Za realan slučaj (segment 90×90 , šablon 30×30) to je razlika između 900 i 4 sabiranja po svakoj od 3.721 pozicija.

2.4. Optimizacija 3: egzaktna celobrojna Q1.31 podela

Rezultat po poziciji prozora je razlomak iz opsega $[0, 1]$, koji se prenosi kao 32-bitna neoznačena vrednost u formatu **Q1.31** (vrednost 1,0 odgovara heksadecimalnom $0x80000000$). ESL model je ovo računao u pokretnom zarezu, uz zaštitnu granicu `if (ncc2 > 1.0) ncc2 = 1.0` koja je bila potrebna zbog grešaka zaokruživanja tipa `double`.

RT model umesto toga izvodi **tačnu celobrojnu podelu** u proširenom registru:

$$NCC^2 = (\text{num_sq} \ll 31) / \text{den_prod}$$

gde su `num_sq` i `den_prod` 52-bitne veličine, pa se deljenik računa u 83 bita. Pošto se ne koristi aritmetika u pokretnom zarezu, greška zaokruživanja ne postoji, a Cauchy-Schwarz-ova nejednakost garantuje $\text{num_sq} \leq \text{den_prod}$ egzaktno — zaštitna granica iz ESL modela matematički otpada i nije implementirana. Degenerisani slučaj konstantnog prozora ili konstantnog šablona (nulta varijansa, deljenje nulom) obrađuje se eksplicitno: rezultat je tada 0.

2.5. Bitska analiza širina signala

Širine svih signala u datapath-u realizovane su u paketu `ncc_pkg.vhd`. Tabela 1 daje njihov pregled.

Veličina	Biti	Tip u <code>ncc_pkg.vhd</code>
piksel slike / šablona	8	<code>pixel_t</code> — unsigned(7 downto 0)
dimenzije <code>img_w/h</code> , <code>tmp_w/h</code>	8	<code>dim_t</code> — unsigned(7 downto 0)
$\bar{f}$, <code>template_mean</code>	8	<code>mean_t</code> — unsigned(7 downto 0)
<code>diff_f</code> , <code>diff_t</code>	9	<code>diff_t</code> — signed (8 downto 0)
<code>sum_num</code>	27	<code>numacc_t</code> — signed (26 downto 0)
<code>sum_den_f</code> , <code>sum_den_t</code>	26	<code>denacc_t</code> — unsigned(25 downto 0)
<code>num_sq</code> , <code>den_prod</code>	52	<code>sq52_t</code> — unsigned(51 downto 0)
rezultat (Q1.31)	32	<code>result_t</code> — unsigned(31 downto 0)
akumulator integralne slike	32	<code>sat_t</code> — unsigned(31 downto 0)

Tabela 1. Bitske širine signala datapath-a i njihova realizacija u VHDL-u

Označenost razlika. Veličine `diff_f`, `diff_t` i `sum_num` su **označene**: razlika piksela i srednje vrednosti je negativna kad god je piksel tamniji od proseka, a suma proizvoda tih razlika menja znak. Broj bita (9 i

27) određen je opsegom vrednosti, a označenost predznakom.

Zašto je datapath bit-egzaktan. Srednje vrednosti $\bar{f}$ i $\bar{t}$ se zaokružuju na ceo broj **pre** oduzimanja (deljenje sa zaokruživanjem: $(\text{suma} + \text{count}/2) / \text{count}$). Time su `diff_f` i `diff_t` egzaktni celi brojevi, pa se kroz desetine hiljada sabiranja u akumulatorima ne akumulira nikakva greška zaokruživanja. Ovo je ključna projektna odluka: ceo datapath je celobrojan, a jedina veličina u formatu sa fiksnim zarezom je konačni rezultat.

2.6. Promenljiva veličina šablona

Dimenzije `tmp_w` i `tmp_h` su parametri koji se zadaju u vreme izvršavanja, preko komandnih registara, a ne konstante u vreme prevođenja. To je neophodno jer se šabloni figura razlikuju: top je 25×15 , dok su kraljica i kralj do 30×30 . Konstante `MAX_IMG_W` = `MAX_IMG_H` = 90 i `MAX_TMP_W` = `MAX_TMP_H` = 30 predstavljaju samo gornju granicu za dimenzionisanje memorija, a ne pretpostavku o fiksnoj veličini.

Ista fleksibilnost omogućava i grubi prolaz *coarse-to-fine* algoritma na sistemskom nivou (segment 45×45 , šablon 15×15), pri čemu samo jezgro ne mora ništa da zna o toj strategiji — ono obrađuje dimenzije koje mu se zadaju.

3. Uklanjanje petlji

ASMD dijagram ne poznaje konstrukte `for` i `while`: svaka petlja mora biti prevedena u oblik `if-goto`, u kome brojači petlji postaju registri, a telo petlje jedno ili više stanja. U algoritmu postoje tri mesta sa petljama i ukupno četiri brojača.

3.1. Izgradnja integralne slike

Dva brojača, `y` i `x`:

```
y = 0
L1: if (y >= img_h) goto L1_END
    row_sum = 0
    x = 0
    L2: if (x >= img_w) goto L2_END
        row_sum = row_sum + image[y*img_w + x]
        SAT[(y+1)*(img_w+1) + (x+1)] = SAT[y*(img_w+1) + (x+1)] + row_sum
        x = x + 1
        goto L2
    L2_END:
        y = y + 1
        goto L1
L1_END:
```

3.2. Srednja vrednost šablona

Jedan brojač, `p`, kao linearni indeks kroz šablon:

```
p = 0; sum_t = 0
L3: if (p >= tmp_w*tmp_h) goto L3_END
    sum_t = sum_t + templ[p]
    p = p + 1
    goto L3
L3_END:
    template_mean = (sum_t + count/2) / count
```

3.3. Glavna petlja

Četiri ugnježdene nivoa: spoljni par (`v`, `u`) prolazi kroz pozicije prozora, unutrašnji par (`y`, `x`) kroz piksele šablona. Brojači `y` i `x` su isti registri kao u odeljku 3.1 — dve faze nisu istovremeno aktivne, pa se resurs deli.


```

    v = 0
L4: if (v >= res_h) goto L4_END
    u = 0
L5: if (u >= res_w) goto L5_END
    sum_f = SAT[(v+th)*(iw+1)+(u+tw)] - SAT[v*(iw+1)+(u+tw)]
        - SAT[(v+th)*(iw+1)+u]      + SAT[v*(iw+1)+u]
    f_bar = (sum_f + count/2) / count
    sum_num = 0; sum_den_f = 0; sum_den_t = 0
    y = 0
L6: if (y >= tmp_h) goto L6_END
    x = 0
L7: if (x >= tmp_w) goto L7_END
    diff_f = image[(v+y)*img_w + (u+x)] - f_bar
    diff_t = templ[y*tmp_w + x]         - template_mean
    sum_num  = sum_num  + diff_f*diff_t
    sum_den_f = sum_den_f + diff_f*diff_f
    sum_den_t = sum_den_t + diff_t*diff_t
    x = x + 1
    goto L7
L7_END:
    y = y + 1
    goto L6
L6_END:
    if (sum_den_f == 0 or sum_den_t == 0)
        result = 0
    else
        result = (sum_num*sum_num << 31) / (sum_den_f*sum_den_t)
    result_map[v*res_w + u] = result
    u = u + 1
    goto L5
L5_END:
    v = v + 1
    goto L4
L4_END:

```

4. Interfejs i okruženje

4.1. Četiri RT interfejsa

RT metodologija zahteva da se za svaki modul definišu četiri grupe signala: komandni, statusni, ulazni podaci i izlazni podaci. Tabela 2 daje njihovo konkretno značenje kod NCC jezgra.

Interfejs	Realizacija
Komandni	Konfiguracioni registri REG_IMG_W/H, REG_TMP_W/H, REG_IMG_ADDR, REG_TMP_ADDR (upisuju se pre pokretanja) i REG_CTRL (upis jedinice pokreće obradu). Na nivou RTL jezgra to su ulazi <code>img_w/h</code> , <code>tmp_w/h</code> i signal <code>start</code> .
Statusni	Registar REG_STATUS (0 = zauzet, 1 = završeno); na nivou jezgra izlazi <code>busy</code> i <code>done</code> .
Ulazni podaci	Ne prenose se preko porta kao vrednosti, već se čitaju iz memorije preko adresno-podatkovnog para: <code>img_addr_o</code> / <code>img_data_i</code> i <code>templ_addr_o</code> / <code>templ_data_i</code> .
Izlazni podaci	Mapa rezultata se upisuje u memoriju preko <code>result_addr_o</code> , <code>result_data_o</code> i <code>result_wr_o</code> ; procesor je čita sa offseta ADDR_RESULTS.

Tabela 2. Realizacija četiri RT interfejsa

Širina izlaznog porta izvedena je funkcionalno iz širina ulaza: pošto su `num_sq` i `den_prod` 52-bitni, a količnik posle skaliranja sa 2^{31} po Cauchy-Schwarz-u nikad ne prelazi 2^{31} , rezultat staje u 32 bita bez zasićenja.

4.2. Registarska mapa

Offset	Registar	Opis	Pristup
0x00	REG_IMG_W	širina segmenta slike	U
0x04	REG_IMG_H	visina segmenta slike	U
0x08	REG_TMP_W	širina šablona	U
0x0C	REG_TMP_H	visina šablona	U
0x10	REG_IMG_ADDR	adresa segmenta u deljenoj memoriji	U
0x14	REG_TMP_ADDR	adresa šablona u deljenoj memoriji	U
0x30	REG_CTRL	upis vrednosti 1 pokreće obradu	U
0x34	REG_STATUS	0 = zauzet, 1 = završeno	Č
0x40	ADDR_RESULTS	početak mape rezultata (32 bita po poziciji)	Č

Tabela 3. Registarska mapa NCC jezgra (preuzeta iz ESL modela)

Adresni prostor programabilne logike, definisan još u ESL modelu, je: deljena memorija na 0x4000_0000, prvo NCC jezgro na 0x5000_0000, drugo na 0x5100_0000 i DMA kontroler na 0x6000_0000. Mapa je od početka projektovana tako da odgovara rasporedu adresa opšte namene na Zynq platformi, pa se može preuzeti neizmenjena pri integraciji u blok dizajn.

4.3. Portovi RTL jezgra

Entitet `ncc_core` ima interfejs prikazan u Tabeli 4. Adresno-podatkovni parovi ponašaju se kao sinhroni memorijski port: adresa se postavlja u jednom taktu, a podatak je validan u narednom.

Port	Smer	Tip	Značenje
<code>clk, rst</code>	in	<code>std_logic</code>	takt i asinhroni reset
<code>start</code>	in	<code>std_logic</code>	pokretanje obrade (jedan takt)
<code>busy, done</code>	out	<code>std_logic</code>	status obrade
<code>img_w, img_h</code>	in	<code>dim_t</code>	dimenzije segmenta
<code>tmp_w, tmp_h</code>	in	<code>dim_t</code>	dimenzije šablona
<code>img_addr_o / img_data_i</code>	out / in	<code>integer / pixel_t</code>	čitanje segmenta
<code>templ_addr_o / templ_data_i</code>	out / in	<code>integer / pixel_t</code>	čitanje šablona
<code>result_addr_o, result_data_o, result_wr_o</code>	out	<code>integer / result_t / std_logic</code>	upis mape rezultata

Tabela 4. Portovi entiteta `ncc_core`

4.4. Memorijsko okruženje

Segment slike i šablon nalaze se u deljenoj memoriji programabilne logike, kojoj pristupa više mastera: procesor (upis konfiguracije), DMA kontroler (prenos segmenta iz spoljne memorije) i oba NCC jezgra. Jezgro je, dakle, **jedan od mastera** na toj memoriji, a ne njen vlasnik. Integralna slika je izuzetak: ona je interna i privatna za jezgro, jer se u fazi izgradnje čita i piše u svakom taktu i ne bi tolerisala deljeni pristup.

5. ASMD dijagram

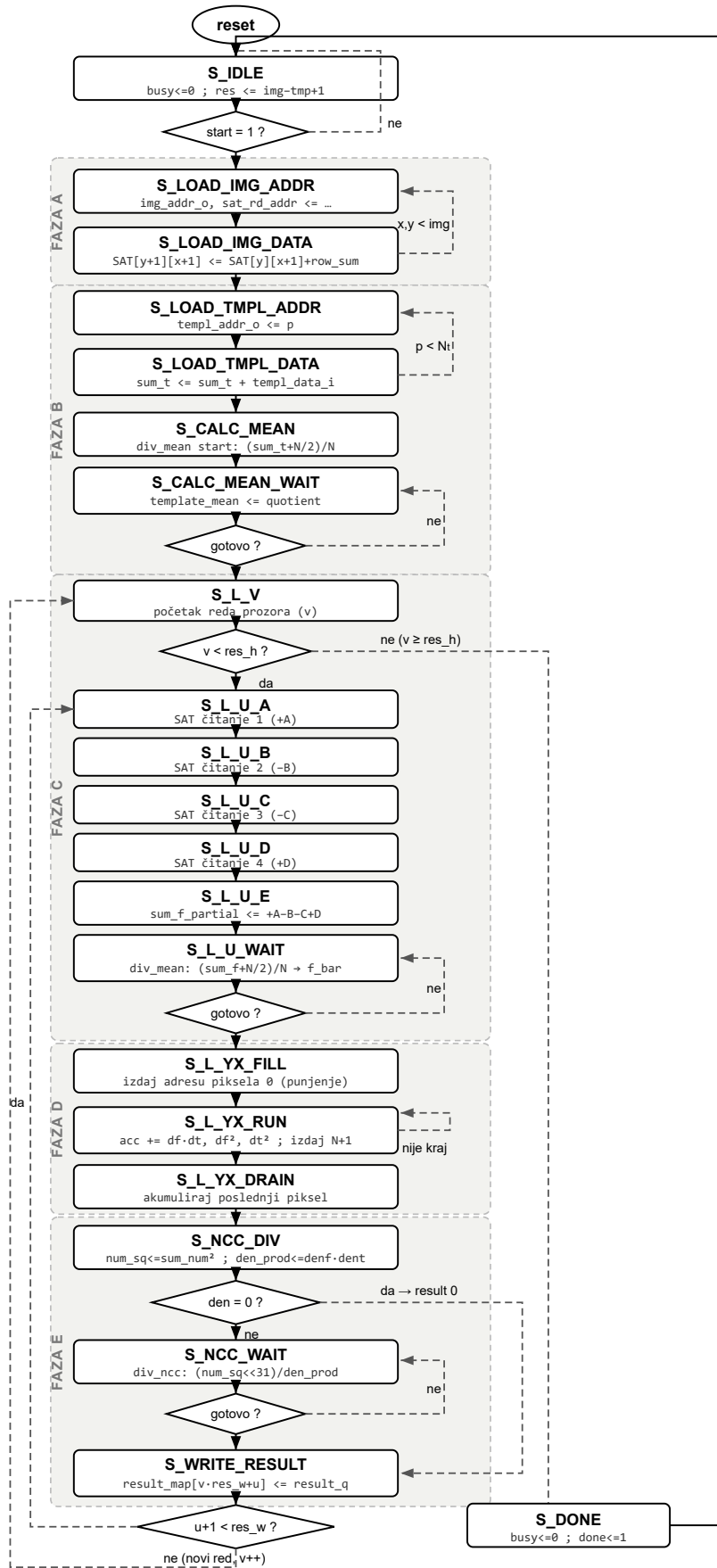
5.1. Pregled stanja

Kontrolna jedinica je Murova mašina stanja sa **21 stanjem**, opisana u dvoprocesnom stilu: jedan proces sadrži isključivo registre (`signal_reg <= signal_next`), drugi svu kombinacionu logiku, sa podrazumevanim dodelama na početku i grananjem po `case state_reg`. Memorija integralne slike izdvojena je u treći, poseban proces. Stanja se prirodno grupišu u pet faza, a sva su — svih 21 — nabrojana u Tabeli 5 i prikazana na ASMD dijagramu (Slika 1).

Faza	Stanja	Broj taktova	Namena
—	S_IDLE	1	čekanje na <code>start</code> , preuzimanje dimenzija, računanje <code>res_w/res_h</code>
A	S_LOAD_IMG_ADDR, S_LOAD_IMG_DATA	$2 \cdot \text{img_w} \cdot \text{img_h}$	čitanje segmenta piksel po piksel i istovremena izgradnja integralne slike
B	S_LOAD_TMPL_ADDR, S_LOAD_TMPL_DATA, S_CALC_MEAN, S_CALC_MEAN_WAIT	$2 \cdot \text{tmp_w} \cdot \text{tmp_h} + \sim 20$	čitanje šablona, sumiranje i deljenje radi dobijanja <code>template_mean</code>
C	S_L_V, S_L_U_A, S_L_U_B, S_L_U_C, S_L_U_D, S_L_U_E, S_L_U_WAIT	$6 + \sim 19$ po prozoru	četiri sekvencijalna čitanja SAT, računanje <code>sum_f</code> i deljenje radi <code>f_bar</code> ; nuliranje akumulatora
D	S_L_YX_FILL, S_L_YX_RUN, S_L_YX_DRAIN	$\text{tmp_w} \cdot \text{tmp_h} + 1$ po prozoru	pipeline-ovana akumulacija tri sume nad pikselima šablona
E	S_NCC_DIV, S_NCC_WAIT, S_WRITE_RESULT	~ 84 po prozoru	kvadriranje, množenje imenilaca, deljenje u 83 bita i upis rezultata
—	S_DONE	1	spuštanje <code>busy</code> , postavljanje <code>done</code>

Tabela 5. *Svih 21 stanje kontrolne jedinice, grupisano po fazama*

Slika 1 prikazuje ASMD dijagram. Pravougaonici su stanja sa pripadajućim RT operacijama, rombovi su uslovni blokovi (grananja), a isprekidane linije povratne grane petlji.



Slika 1. ASMD dijagram kontrolne jedinice NCC jezgra (svih 21 stanje, pet faza)

5.2. Faza A: učitavanje slike i izgradnja SAT

Segment se čita piksel po piksel, a integralna slika se gradi **u istom prolazu** — nije potrebna zasebna faza. Par stanja postoji zbog kašnjenja sinhronne memorije: u stanju `S_LOAD_IMG_ADDR` izdaju se adresa piksela i adresa elementa integralne slike iznad tekućeg, a u stanju `S_LOAD_IMG_DATA` oba podatka su validna, pa se računa nova vrednost `row_sum` i upisuje novi element integralne slike. Cena faze je dva takta po pikselu, odnosno 16.200 taktova za segment 90×90 .

5.3. Faza B: šablon i njegova srednja vrednost

Šablon se čita linearno, po istom obrascu adresa/podatak, uz akumulaciju sume. Po završetku se pokreće sekvencijalni delilac koji računa $\text{template_mean} = (\text{sum_t} + N/2) / N$, gde je $N = \text{tmp_w} \cdot \text{tmp_h}$. Deljenje sa zaokruživanjem je namerno: zaokružena celobrojna srednja vrednost je preduslov za bit-egzaktan datapath (odjeljak 2.5). Ova faza se izvršava jednom po pozivu.

5.4. Faza C: priprema prozora

Za svaku poziciju prozora treba pročitati četiri elementa integralne slike. Pošto je integralna slika realizovana kao blok-memorija sa jednim portom, četiri čitanja **ne mogu** biti istovremena — razložena su u pet stanja (`S_L_U_A` do `S_L_U_E`), pri čemu svako stanje izdaje sledeću adresu i istovremeno akumulira podatak koji je stigao po prethodnoj. U petom stanju se, uz poslednji sabirak, pokreće delilac za `f_bar` i nuliraju sva tri akumulatora, a `S_L_U_WAIT` čeka rezultat deljenja.

5.5. Faza D: pipeline-ovana MAC petlja

Ovo je unutrašnja petlja i ubedljivo najskuplji deo obrade: izvršava se $\text{tmp_w} \cdot \text{tmp_h}$ puta po svakoj poziciji prozora, dakle preko 3,3 miliona puta po pozivu za pun segment. Naivna realizacija bi po pikselu trošila dva takta (jedan za adresu, jedan za podatak). Umesto toga, faza je organizovana protočno: u stanju `S_L_YX_RUN` se **istovremeno** akumulira podatak piksela N i izdaje adresa piksela $N + 1$. Stanje `S_L_YX_FILL` puni protočnu strukturu (izdaje prvu adresu bez akumulacije), a `S_L_YX_DRAIN` je prazni (akumulira poslednji podatak bez izdavanja nove adrese). Cena je time **jedan takt po pikselu** plus jedan takt režije po prozoru. Slika 2 prikazuje preklapanje.

takt	t0	t1	t2	t3	...	tN-1	tN
stanje	FILL	RUN (ponavlja se N-1 puta)				RUN	DRAIN
izdaje adresu	px 0	px 1	px 2	px 3	...	px N-1	—
akumulira	—	px 0	px 1	px 2	...	px N-2	px N-1

Ukupno: $N + 1$ takta za N piksela (jedan takt po pikselu + jedan takt režije).

Bez protočnosti bi bilo $2N$ taktova — ušteda blizu 50 % ukupne latencije jezgra.

Slika 2. Preklapanje izdavanja adrese i akumulacije u pipeline-ovanoj unutrašnjoj petlji

5.6. Faza E: normalizacija i upis rezultata

U stanju `S_NCC_DIV` se najpre proverava degenerisani slučaj: ako je bilo koja od suma kvadrata jednaka nuli, rezultat je 0 i deljenje se preskače. Inače se kombinaciono računaju $\text{num_sq} = \text{sum_num}^2$ i $\text{den_prod} = \text{sum_den_f} \cdot \text{sum_den_t}$, deljenik se pomera za 31 mesto ulevo i pokreće se 83-bitni delilac. Stanje `S_NCC_WAIT` čeka njegov signal završetka, a `S_WRITE_RESULT` upisuje rezultat u mapu i pomera brojače prozora.

6. Datapath i controlpath

6.1. Registri

Registri su identifikovani iz oblika `if-goto` (poglavlje 3), grupisanjem po odredišnoj promenljivoj. Tabela 6 daje pregled.

Registar	Širina	Upisuje se u fazi
<code>img_w, img_h, tmp_w, tmp_h, res_w, res_h</code>	8	S_IDLE (na start)
<code>y, x</code> — deljeni: izgradnja SAT i unutrašnja petlja	7	A i D
<code>v, u</code> — pozicija prozora	7	C i E
<code>p</code> — linearni indeks šablona	10	B
<code>row_sum, sum_t, sum_f_partial</code>	32	A, B, C
<code>template_mean, f_bar</code>	8	B, C
<code>sum_num</code> (označen)	27	D (akumulator)
<code>sum_den_f, sum_den_t</code>	26	D (akumulatori)
<code>result_q</code>	32	E
<code>busy, done</code>	1	S_IDLE, S_DONE

Tabela 6. Registri datapath-a

6.2. Funkcionalne jedinice

Datapath sadrži tri paralelna množača u unutrašnjoj petlji ($\text{diff}_f \cdot \text{diff}_t$, diff_f^2 , diff_t^2), koji se preslikavaju u DSP48E1 blokove, kao i dva množača van petlje (kvadriranje brojioca i proizvod imenilaca). Deljenje resursa nije primenjeno na tri množača unutrašnje petlje: pošto se ta petlja izvršava milionima puta, serijalizacija bi utrošila najskuplji deo obrade radi uštede od svega nekoliko DSP blokova, kojih ionako ima dovoljno (odjeljak 7.2).

6.3. Sekvencijalni delilac

Deljenje promenljivim deliocem je najskuplja operacija u datapath-u. Realizovano je kao zaseban parametrizovan modul `seq_divider`, po klasičnom algoritmu sa vraćanjem (engl. *restoring division*): u svakom taktu se ostatak pomera za jedno mesto, poredi sa deliocem i po potrebi umanjuje, čime se dobija jedan bit količnika. Kritična putanja po taktu je time svedena na jedno oduzimanje i jedno poređenje.

Modul se instancira dva puta, sa različitim parametrom širine:

Instanca	Širina	Namena	Taktova po deljenju	Poziva po pozivu jezgra
<code>div_mean</code>	18	<code>template_mean</code> i <code>f_bar</code>	~19	$1 + \text{res_w} \cdot \text{res_h}$
<code>div_ncc</code>	83	$\text{NCC}^2 = (\text{num_sq} \ll 31) / \text{den_prod}$	~83	$\text{res_w} \cdot \text{res_h}$

Tabela 7. Instance sekvencijalnog delioca

Instanca `div_mean` deljena je između dve namene (srednja vrednost šablona i srednja vrednost prozora) jer se te dve operacije nikada ne izvršavaju istovremeno — primer deljenja resursa koje ništa ne košta.

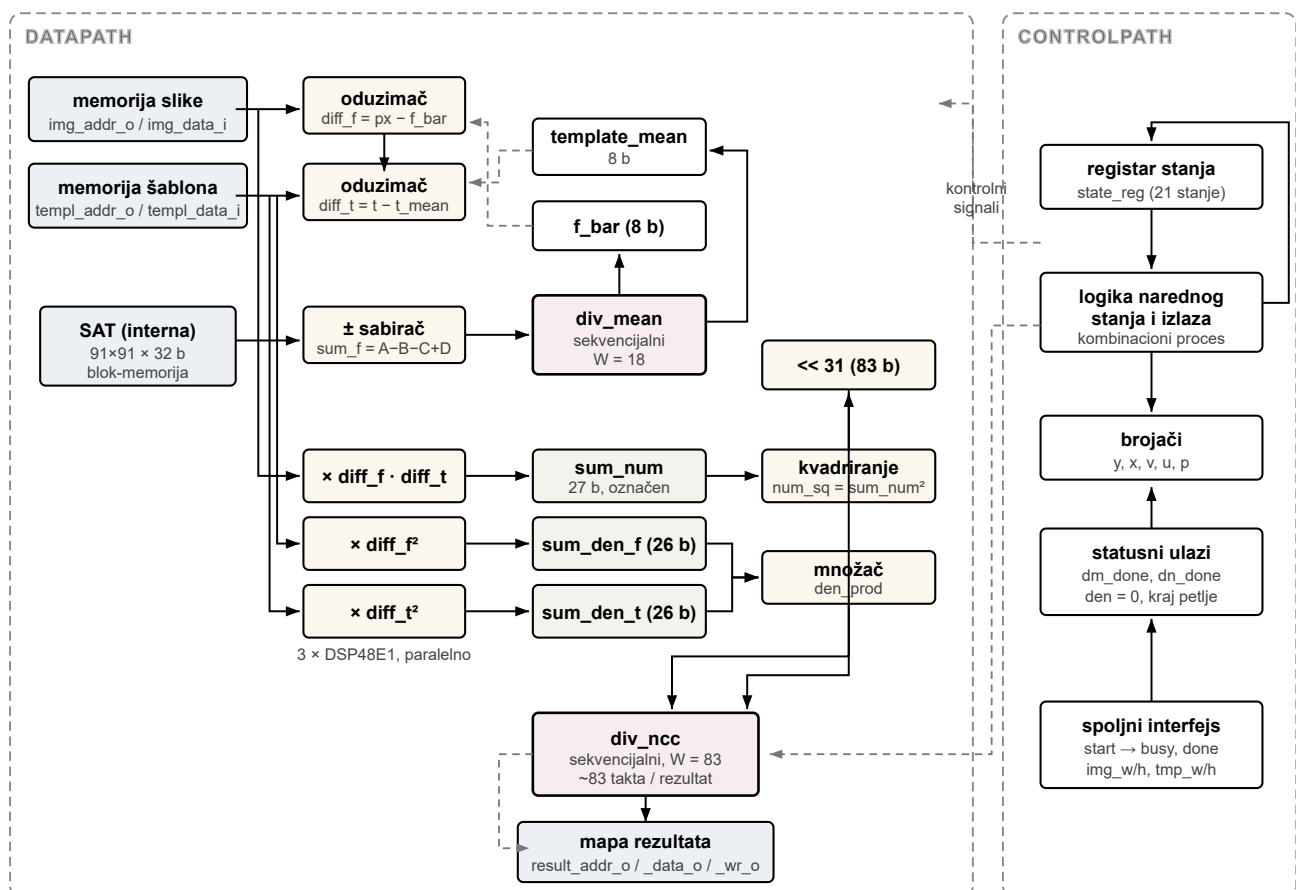
Komunikacija sa deliocem je preko rukovanja: kontrolna jedinica postavlja operande i generiše impuls start, zatim čeka signal done u posebnom stanju čekanja.

6.4. Memorije

Memorija	Veličina	Pristup	Realizacija
segment slike	$90 \times 90 \times 8$ b	čitanje	spoljna, deljena (procesor / DMA / oba jezgra)
šablon	$30 \times 30 \times 8$ b	čitanje	ista spoljna memorija, druga adresna oblast
integralna slika (SAT)	$91 \times 91 \times 32$ b	čitanje i upis	interna , privatna, blok-memorija
mapa rezultata	do $90 \times 90 \times 32$ b	upis (jezgro), čitanje (procesor)	spoljna, offset ADDR_RESULTS

Tabela 8. Memorije u sistemu

6.5. Blok dijagram



Slika 3. Blok dijagram datapath-a i controlpath-a NCC jezgra

Podela je klasična: controlpath je isključivo mašina stanja sa brojačima, koja prema datapath-u šalje upravljačke signale (izbor operanada, dozvole upisa, impulse za pokretanje delilaca), a od njega prima statusne signale (završetak deljenja, nulta varijansa, kraj petlji). Datapath ne sadrži nikakvu odluku o toku izvršavanja.

7. Analiza posle sinteze

7.1. Metod

Analiza je sprovedena nad opisom u fajlovima `ncc_pkg.vhd` i `ncc_core.vhd`, koji zajedno čine ceo dizajn (modul `seq_divider` nalazi se u istom fajlu kao i jezgro).

- **Sinteza i vremenska analiza:** Vivado 2025.2, komanda `synth_design -top ncc_core -part xc7z010clg225-2`, zatim `create_clock -period 10.000` i izveštaji `report_utilization` i `report_timing_summary`.
- **Funkcionalna provera i merenje latencije:** XSim, simulacija ponašanja nad stvarnim podacima; testbench broji takte u kojima je signal `busy` aktivan.

7.2. Utrošeni resursi

Resurs	Utrošeno	Raspoloživo	Iskorišćenost
Slice LUT (sve kao logika)	1.526	17.600	8,67 %
Slice registri (FF)	554	35.200	1,57 %
DSP48E1	9	80	11,25 %
Block RAM (RAMB36E1)	9	60	15,00 %

Tabela 9. *Utrošeni resursi posle sinteze (xc7z010-clg225-2)*

Jezgro zauzima manje od devetine logičkih resursa uređaja. To je i bio cilj: sistemski model predviđa dve paralelne instance, a pored njih u programabilnoj logici moraju stati i AXI interkonekt, kontroler blok-memorije i DMA. Sa 8,67 % po instanci, dve instance troše oko 17 % LUT-ova i ostavljaju veliku rezervu.

7.3. Kritična putanja i maksimalna frekvencija

Veličina	Vrednost
Zadati period takta	10,000 ns (100 MHz)
Najgora vremenska rezerva (WNS)	+1,179 ns
Kašnjenje kritične putanje	8,670 ns (logika 6,860 ns · rutiranje 1,810 ns)
Broj nivoa logike na kritičnoj putanji	12 (8×CARRY4, 2×DSP48E1, 2×LUT)
Maksimalna frekvencija	~113 MHz (period 8,821 ns)
Najgora rezerva za zadržavanje (WHS)	+0,127 ns
Krajnjih tačaka koje krše ograničenje	0 od 1.445

Tabela 10. *Rezultati vremenske analize pri periodu takta od 10 ns*

Dizajn zatvara ograničenje od 100 MHz sa rezervom od 1,179 ns, uz pozitivnu rezervu i za zadržavanje. Nijedna od 1.445 krajnjih tačaka ne krši ograničenje. Ostvariva frekvencija je približno 113 MHz.

Kritična putanja vodi od akumulatora `sum_num_reg[16]` do radnog registra `delioca_div_ncc/work_reg[78]`, dakle kroz kvadriranje brojioca i ulaz u 83-bitni delilac. Da je kritična putanja bila

unutar same unutrašnje petlje, jedini lek bio bi dodatni protočni stepen; ovako je ona na putanji koja se izvršava jednom po poziciji prozora, pa bi se po potrebi mogla registrovati bez uticaja na propusnost.

Ovaj rezultat je posledica jedne projektne odluke: sva tri deljenja izvode se sekvencijalno, jedan bit količnika po taktu. Kombinaciono deljenje promenljivim deliocem davalo bi vremensku rezervu od približno -46 ns, dakle sintetizovano kolo ne bi radilo ni na 20 MHz. Sekvencijalna realizacija svodi kritičnu putanju delioca na jedno oduzimanje i jedno poređenje, po cenu latencije koja se, kao što pokazuje sledeći odeljak, u ukupnom bilansu jedva primećuje.

7.4. Latencija i propusnost

Latencija je izmerena simulacijom na stvarnim podacima: gornji levi segment 90×90 slike board2.txt (polje a8 šahovske table) i šablon crnog topa dimenzija 25×15 . Mapa rezultata je 66×76 , dakle 5.016 pozicija prozora.

Veličina	Vrednost
Latencija po pozivu (izmereno)	2.451.212 taktova
Vreme po pozivu pri 100 MHz	24,51 ms
Broj obrađenih pozicija prozora	5.016
Vreme po poziciji prozora	4,89 μ s
Propusnost	~ 204.600 pozicija/s
Taktova po piksel-operaciji	1,30

Tabela 11. Izmerena latencija i izvedena propusnost

Terminologija. Pod *latencijom* se podrazumeva broj taktova od postavljanja signala start do signala done za jedan poziv jezgra; pod *propusnošću* količina posla po jedinici vremena, ovde izražena brojem obrađenih pozicija prozora u sekundi. Pošto jezgro obrađuje jedan poziv u potpunosti pre nego što prihvati sledeći, ove dve veličine su uzajamno recipročne. Konkretno, propusnost = $5.016 \text{ pozicija} / 24,51 \text{ ms} \approx 204.600 \text{ pozicija/s}$ (odnosno $1 / 4,89 \mu\text{s}$), a takta po piksel-operaciji = $2.451.212 / (5.016 \cdot 375) = 1,30$.

Izmerena latencija se dobro poklapa sa analitičkim modelom. Trošak po poziciji prozora iznosi:

$$T_{\text{prozor}} = tmp_w \cdot tmp_h + 110 \text{ taktova}$$

gde sabirak 110 čine: pet taktova za čitanje integralne slike, oko 19 taktova za deljenje pri računanju f_bar , dva takta za punjenje i pražnjenje protočne strukture, oko 83 takta za 83-bitno deljenje i jedan takt za upis rezultata. Za šablon 25×15 to je 485 taktova po prozoru, odnosno 2.432.760 taktova za svih 5.016 pozicija. Uz fazu A (16.200) i fazu B (769), model predviđa 2.449.729 taktova, dok je izmereno 2.451.212 — odstupanje je 1.483 takta, odnosno **0,06 %**. Model se, dakle, može koristiti za predviđanje latencije pri drugim veličinama segmenta i šablona.

Ukupna latencija je zbir tri člana:

Faza	Model	Taktova	Udeo
A — učitavanje slike i izgradnja SAT	$2 \cdot \text{img_w} \cdot \text{img_h} = 2 \cdot 90 \cdot 90$	16.200	0,7 %
B — učitavanje šablona i njegova srednja vrednost	$2 \cdot N + 19 \quad (N = 375)$	769	< 0,1 %
C, D, E — glavna petlja	$\text{res_w} \cdot \text{res_h} \cdot (N+110) = 66 \cdot 76 \cdot 485$	2.432.760	99,2 %
ostatak (prelazi S_L_V , zaokruženja modela)	—	1.483	0,1 %
Ukupno (izmereno)	—	2.451.212	100 %

Tabela 12. Raščlanjenje latencije po fazama (šablon 25×15)

Dominacija glavne petlje potvrđuje da je protočnost unutrašnje petlje bila prava meta optimizacije: ona neposredno određuje član N u izrazu iznad, koji za realne veličine šablona nosi 77 % troška po prozoru.

7.5. Funkcionalna verifikacija

Ispravnost je proverena na dva nivoa, oba poređenjem sa referentnim vrednostima iz validiranog modela u jeziku C (Korak 1), koji je nezavisno proveren nad celom tablom — 32 od 32 figure ispravno prepoznate, uz FEN notaciju identičnu zvaničnoj.

Test	Podaci	Očekivano	Ishod
ncc_core_tb	sintetički segment 4×4 , šablon 2×2	svih 9 pozicija prozora bit-identično referentnom modelu, uključujući oba ekstrema (0x00000000 i 0x80000000)	prolazi (9 / 9)
ncc_core_real_tb	stvarni segment 90×90 (polje a8) i šablon crnog topa 25×15	maksimum 0x80000000 na poziciji (u = 32, v = 14)	prolazi (0x80000000 @ 32,14)

Tabela 13. Rezultati funkcionalne verifikacije

Sintetički test je namerno konstruisan tako da šablon bude tačno ugrađen u sliku na poziciji (1, 1): time se u istom testu pokrivaju i tačna nula i tačan maksimum 2^{31} , a svih devet vrednosti se može proveriti ručno. Test nad stvarnim podacima potvrđuje ponašanje na punoj veličini problema i na podacima koji nisu konstruisani: dobijeni maksimum 0x80000000 odgovara vrednosti $NCC^2 = 1,0$, što je daleko iznad praga 0,5625 (0x48000000), a položaj maksimuma odgovara polju koje u zvaničnoj FEN notaciji zaista sadrži crnog topa.

8. Zaključak

RT model NCC jezgra je opisan u VHDL-u, funkcionalno verifikovan i sintetizovan za ciljni uređaj xc7z010-c1g225-2. Ključni rezultati:

- rezultat je **bit-identičan** referentnom modelu u jeziku C, na sintetičkom i na stvarnim podacima (odeljak 7.5);
- dizajn **zatvara vremenska ograničenja** pri 100 MHz, sa rezervom od 1,179 ns i ostvarivom frekvencijom od približno 113 MHz (odeljak 7.3);
- zauzeće resursa (8,67 % LUT, 11,25 % DSP, 15 % blok-memorije) ostavlja dovoljno prostora za drugu instancu jezgra i za AXI infrastrukturu, što je preduslov za paralelizaciju predviđenu sistemskim modelom (odeljak 7.2);
- propusnost iznosi približno 204.600 pozicija prozora u sekundi pri taktu od 100 MHz, odnosno 1,30 takta po piksel-operaciji (odeljak 7.4).

Time su ispunjeni svi uslovi da se jezgro spakuje u IP sa AXI interfejsom i integriše u sistem na programabilnoj logici.